

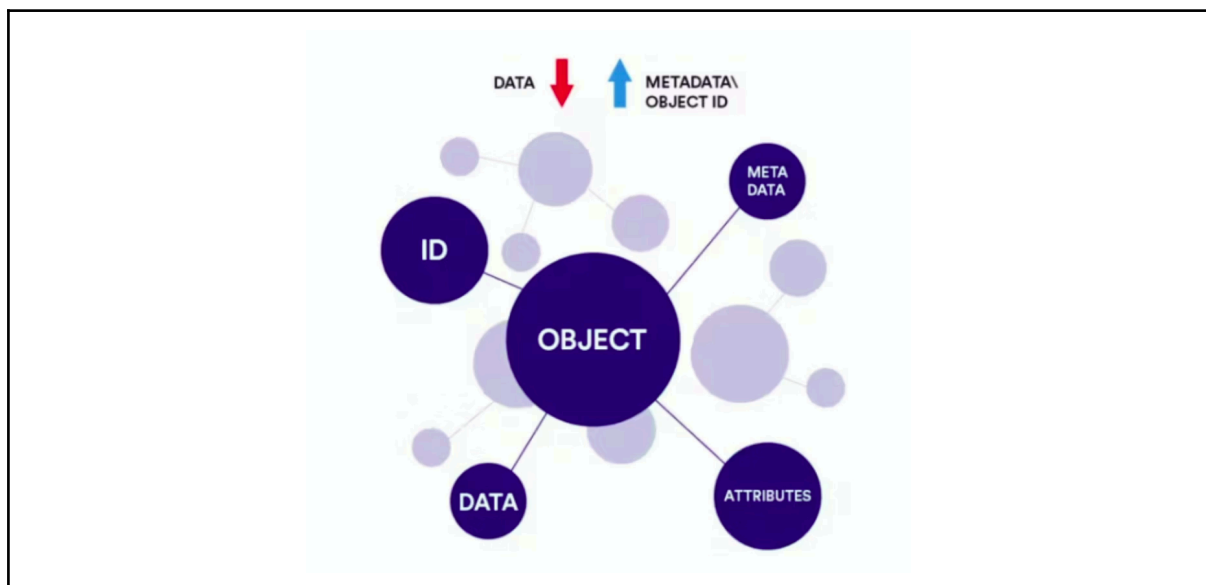
Section 7: Storage in System Design

Object Storage in Modern Systems

In this lecture, we will explore object storage, the foundation of the modern cloud data platform, and understand why services like S3, Blob Storage, and Google Cloud Services have become the default choice for strong data at massive scale.

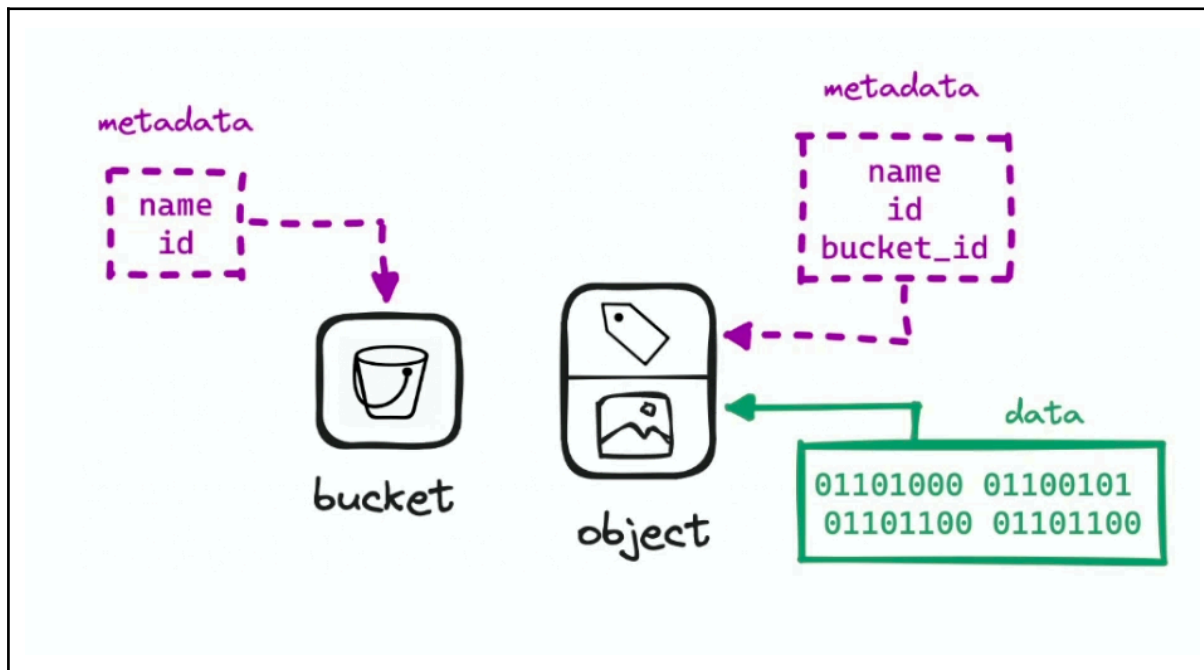
What is Object Storage?

The idea is that data is stored as self-contained objects rather than files organized in folders or blocks attached to disks. Every object combines the actual data, a unique identifier used for retrieval, and rich metadata that describes the object and how it should be managed. This design becomes incredibly powerful at scale because the storage system no longer depends on traditional directory structure or fixed volumes.



Instead, objects live in a flat namespace and can be distributed across thousands of servers transparently. That is why object storage has become the default choice for storing massive amounts of unstructured data, like images, videos, documents, backups, logs and data lake content. Services like Amazon S3 demonstrate this model at enormous scale, allowing organisations to store billions of objects while maintaining high durability, availability, and virtually unlimited growth.

Key Concepts in Object Storage

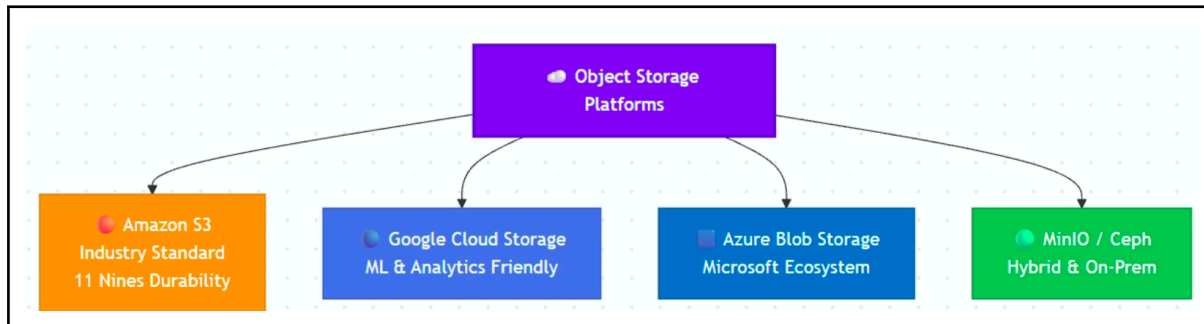


To work effectively with object storage, there are 3 core concepts that every architect should understand - objects, buckets and metadata.

- **Objects:** The object is the fundamental building block. Unlike traditional storage systems that separate data from its attributes, an object is a self-contained package that stores data together with its identifier and metadata. This makes objects portable, independent, and easy to manage at massive scale.
- **Buckets:** Objects are organized inside buckets, which acts as a logical container. Buckets provide a simple way to group and manage related data without the complexity of deep directory hierarchies. In cloud platforms such as Amazon S3, every object belongs to a bucket and a single bucket can store an enormous number of objects.
- **Metadata:** What truly differentiates object storage is metadata. Beyond basic information like content type, size and timestamp, metadata can hold application-specific attributes like ownership, project information, retention policies, or access classification. This additional context allows applications to search, automate, secure, and manage data much more intelligently.

Together, objects, buckets and metadata form the foundation that enables object storage systems to scale efficiently while remaining flexible and easy to operate.

Popular Object Storage Platforms

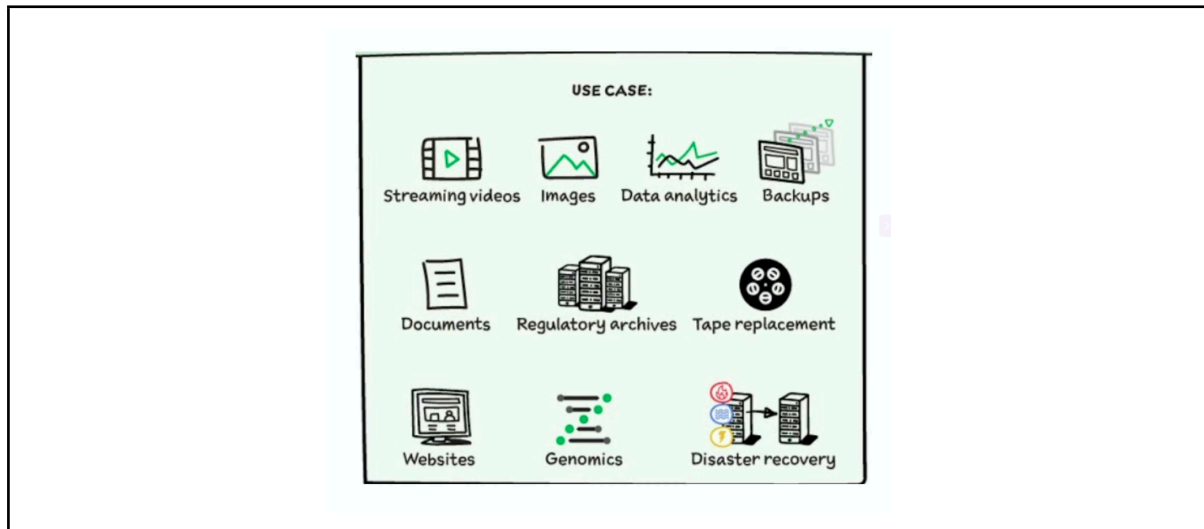


As object storage became the backbone of cloud-native applications, several platforms emerged, each optimized for different ecosystem and operational needs.

- **Amazon S3:** Amazon S3 is widely considered the industry benchmark. Beyond its massive scale and durability, what makes S3 so influential is that it effectively defines many of the patterns and APIs that other object storage platforms now follow. From backups and media storage to analytics and data lakes, it's used across virtually every industry.
- **Google Cloud Storage:** GCS takes a similar approach but focuses on simplicity and deep integration with Google's data and AI ecosystem. Organizations heavily invested in analytics, machine learning, and big data pipelines often find it a more natural fit.
- **Azure Blob Storage:** Azure Blob Storage is particularly attractive for Microsoft centric environments. If your applications already rely on Azure services, .NET technologies, or enterprise Microsoft tooling, Blob Storage provides a seamless operational experience with strong governance and security capabilities.
- **On-prem alternatives (MinIO and Ceph):** Not every organization wants all of its data in a public cloud. Platforms like MinIO and Ceph bring object storage into private data centers and hybrid environments, offering S3-compatible APIs while giving teams greater control over infrastructure and data residency.

As architects, the key takeaway is that the core object storage concepts remain largely the same across all platforms. The real decision factors are ecosystem alignment, operational requirements, compliance requirements, and how closely the storage platforms integrate with the rest of your architecture.

Common Use Cases



The real strength of object storage becomes clear when we look at where it is used in production systems.

- One of the most common use cases is storing media content. Applications that handle images, videos, documents, or user uploads often need storage that can grow almost indefinitely without requiring complex capacity planning. Object storage was designed specifically for that challenge.
- It is also the foundation of modern backup and archival strategies. Since durability is prioritized over low latency access, organisations can store years of backups, their compliance records, and their disaster recovery data at a relatively lower cost.
- In the world of analytics, object storage serves as the backbone of data lake. Instead of loading data directly into databases, companies often store raw datasets in the object storage first, allowing analytics platforms, data warehouses, and AI systems to process them whenever needed.
- Object storage can serve files directly over HTTP, it becomes a simple and very cost-effective way to host frontend applications, document sites, landing pages, and content portals without managing web servers.
- Object storage plays a very critical role in IoT and machine learning pipelines. Whether it is sensor reading arriving from millions of devices or terabytes of training data for AI models, object storage provides a scalable landing zone that can absorb massive volumes of data and feed downstream processing systems.

A useful rule of thumb is this. Whenever you are dealing with large-scale unstructured data that needs durability, scalability, and cost efficiency, object storage is usually the first storage option architects consider.

Important Considerations with Object Storage

As architects, it is very important to understand that object storage is optimized for scalability and durability, not for every possible workload. Choosing it effectively means understanding both its strengths and its trade-off.

From a performance perspective, object storage behaves very differently from block or file storage. Accessing data typically involves network calls and APIs, so latency is naturally higher. That's perfectly acceptable for media files, backups, analytics, datasets, and similar workloads. But it is usually not the right choice for databases or latency-sensitive applications.

Where object storage excels is throughput and scale. These platforms are designed to handle enormous amounts of parallel access across thousands of clients, which is why they work so well for data lakes, content delivery, and large-scale processing pipelines.

Another very important consideration is consistency. While modern platforms have improved significantly, architects should always understand the consistency guaranteed offered by their chosen cloud provider and design applications accordingly. Storage semantics can influence how quickly newly written data becomes visible across distributed systems.

Access pattern matter as well. Object storage performs best when data is written once and read many times. Frequent updates, random modifications, or append-heavy workloads are often better served by file or block storage.

Then finally cost is another area where architectural decisions have major impact. Cloud providers offer multiple storage classes, ranging from high performance tiers for active data to archive tiers optimized for long-term retention. Matching data to the appropriate storage class can dramatically reduce cost without affecting business requirements. It is also very important to remember that cloud storage costs extend beyond capacity. Request charges, data retrieval fees, and network egress costs can sometimes exceed storage costs if workloads are not carefully designed.

A common practice is to automate the lifecycle management. As data ages and access patterns change, objects can be moved automatically from standard storage to cheaper archival tiers, or deleted when they are no longer needed. Combined with regular monitoring and usage analysis, this will ensure that object storage remains both operationally efficient and cost-effective at scale. So the key takeaway is simple. Object storage scales remarkably well, but successful architectures optimize not just for capacity, but also for access pattern, consistency requirements, and long-term cost management.

Summary & Key Takeaways

- Object storage is essential for modern, scalable, unstructured data storage
- Understand buckets, objects, and metadata
- Choose the right storage based on access patterns and cost
- Embrace object storage for media, backups, data lakes, and more
- What's next:
 - File Systems and Distributed Storage

Interview Questions - Object Storage

- **Conceptual Questions**
 - What is object storage and how is it different from file or block storage?
 - When would you choose object storage over a traditional file system?
 - Explain the structure of an object in object storage. What role does metadata play?
- **System Design Scenarios**
 - Design a media hosting platform (e.g., YouTube). How would you use object storage for video uploads and streaming?
 - How would you architect a cost-efficient backup system for petabytes of logs using Amazon S3?
 - In a microservices system, how would services securely share large files using object storage?
- **Practical & Trade-off Questions**
 - What are the performance trade-offs of using object storage for real-time access?
 - How do storage class tiers (e.g., S3 Standard vs. Glacier) influence design decisions?
 - How would you implement access control for user-specific data stored in object storage?